```
lemmatizer = WordNetLemmatizer() lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]
print("Lemmatized Words:", lemmatized_words)
```

Key Features of NLTK:

1. **Tokenization**: Splits text into words and sentences, enabling finer granularity for analysis.
2. **Stopword Removal**: Filters out common words like "the," "is," "in," etc., that do not add significant value to the context.
3. **Stemming**: Reduces words to their root forms (e.g., "running" to "run"), simplifying text while retaining its meaning.
4. **Lemmatization**: Converts words to their base forms (e.g., "better" to "good"), offering a more linguistically accurate representation compared to stemming.
5. **Extensive Datasets**: Includes corpora like WordNet and tools for linguistic analysis, making it a go-to library for educational and research purposes.

spaCy

spaCy is a fast, production-ready NLP library designed for efficient processing of large text datasets. It is particularly strong in tasks such as named entity recognition (NER), dependency parsing, and part-of-speech tagging. Unlike NLTK, spaCy focuses on modern NLP workflows, emphasizing performance and scalability. This makes it a preferred choice for developers and data scientists working on industrial applications or real-time systems.

Example: Text Processing with spaCy

```
import spacy

# Load spaCy model
nlp = spacy.load("en_core_web_sm") text = "Natural Language Processing enables machines to understand human language."

doc = nlp(text)

# Tokenization
print("Tokens:", [token.text for token in doc]) # Part-of-Speech Tagging
print("POS Tags:", [(token.text, token.pos_) for token in doc]) # Named Entity Recognition
print("Entities:", [(ent.text, ent.label_) for ent in doc.ents])
```